Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

06
인공 신경망

직선 하나로는 풀 수 없던 문제를, 뉴런을 여러 층 쌓아서 푸는 방법을 배웁니다. 딥러닝의 "딥(deep)"이 바로 이 층입니다.

오늘 다루는 것

1. 머신 러닝 기본 개념 정리하기
2. 퍼셉트론
3. XOR 문제: 단층 퍼셉트론과 다층 퍼셉트론
4. 역전파
5. 다층 퍼셉트론으로 손글씨 분류하기
6. 다층 퍼셉트론으로 MNIST 분류하기
7. 과적합을 막는 방법들
8. 기울기 소실과 폭주

끝나고 나면 할 수 있는 것

1. 층을 여러 개 쌓아야 하는 이유를 XOR 예시로 설명할 수 있다.
2. 순전파와 역전파가 각각 무엇을 하는지 그림으로 설명할 수 있다.
3. 파이토치로 손글씨 숫자를 분류하는 신경망을 직접 학습시킬 수 있다.

2026년 10월 7일 수요일 14:30 (10/7 자료 1/3)
발표자 설명, 코랩 실습, 퀴즈 돌아가며 풀기 순서로 진행합니다. 막힌 질문은 디스코드에 남겨 주세요.

06

인공 신경망

03~05장에서는 입력에 가중치를 곱해 더하는 **한 층짜리 모델**로 예측했습니다. 하지만 세상의 문제는 대부분 직선 하나로 나눌 수 없을 만큼 복잡합니다. 이 장에서는 작은 계산 단위(뉴런)를 여러 층으로 쌓아서 복잡한 문제를 푸는 인공 신경망을 만들고, 층이 깊어질 때 생기는 문제와 해결 방법까지 살펴봅니다.

왜 배우나: 오늘의 학습 경로



06-01은 앞으로 계속 나올 용어(분류, 정확도, 활성화 함수 등)를 정리하는 준비 운동입니다. 06-02~06-04가 이 경로의 본체이고, 06-05~06-06에서 손글씨를 분류해 보며, 06-07~06-08은 깊은 신경망을 잘 학습시키는 요령입니다.

06-01 머신 러닝 기본 개념 정리하기

핵심 한 줄

모델은 분류나 회귀를 하고, 정확도·정밀도·재현율로 평가하며, 활성화 함수로 직선이 아닌 관계를 표현한다.

왜 배우나

신경망을 만들기 전에, 앞으로 계속 나올 말들을 한 번에 정리해 둡니다. "이 모델은 분류 모델이고, 출력층에 소프트맥스를 쓰고, 정확도는 95%다" 같은 문장을 읽을 수 있어야 뒤의 실습 코드가 눈에 들어옵니다.

1) 무엇을 맞히나: 분류와 회귀

종류	맞히는 것	예
이진 분류	둘 중 하나	스팸 / 정상 메일
다중 클래스 분류	여러 개 중 하나	손글씨 숫자 0~9
회귀	연속된 숫자	공부 시간으로 점수 예측 (03장)

정답을 알려 주고 배우게 하면 **지도 학습**, 정답 없이 데이터의 구조를 찾게 하면 **비지도 학습**, 행동의 결과로 보상을 받으며 배우면 **강화 학습**입니다. 이 책은 대부분 지도 학습을 다룹니다. 데이터 한 줄(학생 한 명, 메일 한 통)은 **샘플**, 그 샘플을 설명하는 각 항목(퀴즈 점수, 단어 수)은 **특성(feature)**이라고 부릅니다.

2) 얼마나 잘했나: 정확도, 정밀도, 재현율

정확도(전체 중 맞힌 비율)만 보면 놓치는 것이 있습니다. 메일 100통 중 스팸이 1통뿐이라면, "전부 정상"이라고만 해도 정확도가 99%입니다. 그래서 맞힌 것과 틀린 것을 네 칸으로 나눈 **혼동 행렬**을 봅니다.



그림 1 실습의 스팸 분류 결과 10통을 네 칸으로 나눈 혼동 행렬. 파랑은 맞힌 것, 빨강은 틀린 것

- **정밀도** $= \frac{TP}{TP + FP}$: "스팸이라고 한 것" 중 진짜 스팸의 비율. 정상 메일을 스팸함에 넣으면 안 될 때 중요합니다.
- **재현율** $= \frac{TP}{TP + FN}$: "진짜 스팸" 중 찾아낸 비율. 하나라도 놓치면 안 될 때(예: 질병 검사) 중요합니다.

비유

공항 보안 검색대

위험물 검사기가 너무 깐깐하면 멀쩡한 가방도 자꾸 열어 보게 되고(정밀도가 낮음), 너무 느슨하면 진짜 위험물을 놓칩니다(재현율이 낮음). 두 숫자는 서로 밀고 당기는 관계라서, 상황에 따라 어느 쪽을 더 중요하게 볼지 정합니다.

가방을 열어 본 것	= 양성이라고 예측
열어 본 가방 중 진짜 위험물 비율	= 정밀도
전체 위험물 중 찾아낸 비율	= 재현율

3) 과적합과 과소 적합

같은 문제집만 반복해서 풀면 문제 번호만 봐도 답을 외우게 되지만, 새 시험에서는 점수가 안 나옵니다. 모델이 훈련 데이터를 외워 버려서 새 데이터에서 성능이 떨어지는 상태가 **과적합**입니다. 반대로 공부가 부족해 훈련 데이터조차 못 맞히면 **과소 적합**입니다. 대책은 06-07에서 자세히 봅니다.

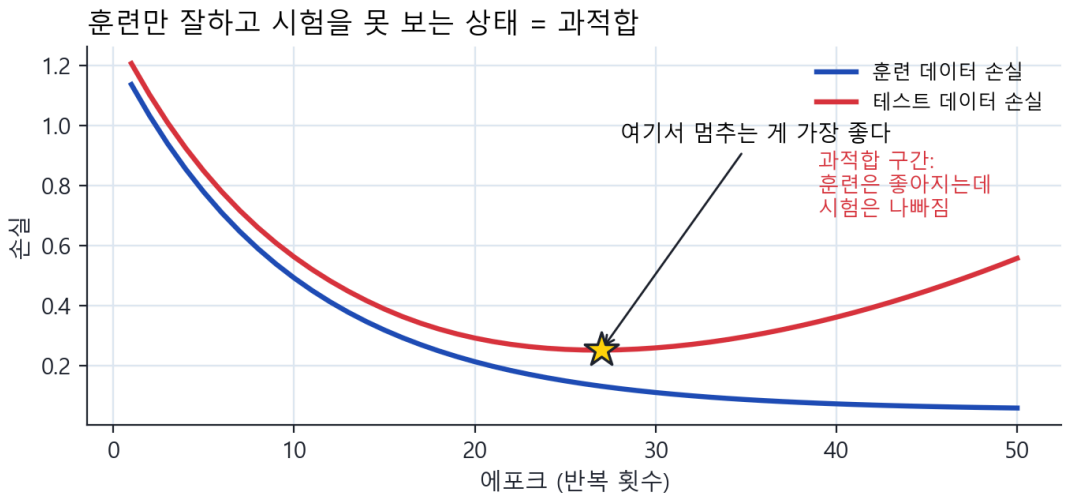


그림 2 훈련 손실은 계속 줄지만 테스트 손실은 어느 순간부터 다시 커진다. 그 직전(별)이 가장 좋은 지점

4) 활성화 함수: 직선을 구부리는 장치

층을 아무리 쌓아도 곱하고 더하기만 하면 결국 직선 하나와 같습니다. 그래서 각 층의 출력을 한 번 **구부려 주는 함수**를 넣는데, 이것이 **활성화 함수**입니다. 은닉층에는 주로 **ReLU**, 이진 분류의 출력층에는 **시그모이드**, 다중 분류의 출력층에는 **소프트맥스**(점수를 합이 1인 확률로 바꿈)를 씁니다.

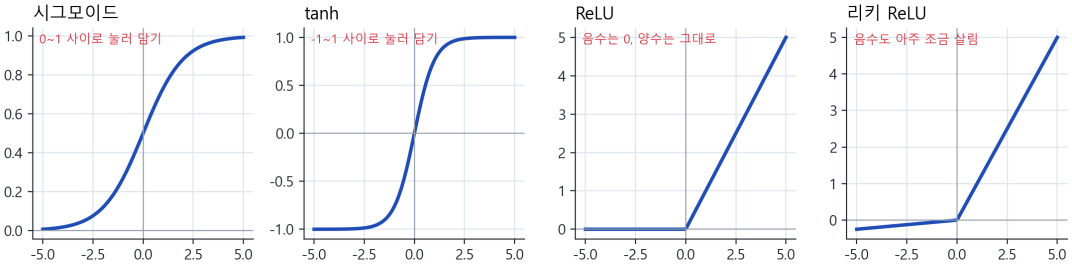


그림 3 대표적인 활성화 함수 네 가지. 모두 곧은 직선이 아니라 어딘가가 꺾이거나 휘어 있다

실습 1 정밀도·재현율 계산과 활성화 함수 비교

```python
import torch

# 1) 스팸 메일 분류기 결과 10통 (1 = 스팸, 0 = 정상)
actual    = [1, 1, 1, 1, 0, 0, 0, 0, 0, 0]
predicted = [1, 1, 1, 0, 1, 0, 0, 0, 0, 0]

TP = sum(a == 1 and p == 1 for a, p in zip(actual, predicted))  # 스팸을 스팸이라고 맞힘
FP = sum(a == 0 and p == 1 for a, p in zip(actual, predicted))  # 정상인데 스팸이라고 함
FN = sum(a == 1 and p == 0 for a, p in zip(actual, predicted))  # 스팸인데 놓침
TN = sum(a == 0 and p == 0 for a, p in zip(actual, predicted))  # 정상을 정상이라고 맞힘
print(f"TP={TP}  FP={FP}  FN={FN}  TN={TN}")
print("정확도:", (TP + TN) / len(actual))
print("정밀도:", TP / (TP + FP), " (스팸이라고 한 것 중 진짜 스팸)")
print("재현율:", TP / (TP + FN), " (진짜 스팸 중 찾아낸 것)")

# 2) 활성화 함수에 같은 값을 넣어 보기
x = torch.tensor([-3.0, -1.0, 0.0, 1.0, 3.0])
print("입력     :", x.tolist())
show = lambda t: [round(v, 2) for v in t.tolist()]   # 소수 둘째 자리까지 보기
print("시그모이드:", show(torch.sigmoid(x)))
print("tanh     :", show(torch.tanh(x)))
print("ReLU     :", show(torch.relu(x)))
print("소프트맥스:", show(torch.softmax(x, dim=0)))
```

실행 결과

```
TP=3  FP=1  FN=1  TN=5
정확도: 0.8
정밀도: 0.75  (스팸이라고 한 것 중 진짜 스팸)
재현율: 0.75  (진짜 스팸 중 찾아낸 것)
입력     : [-3.0, -1.0, 0.0, 1.0, 3.0]
시그모이드: [0.05, 0.27, 0.5, 0.73, 0.95]
tanh     : [-1.0, -0.76, 0.0, 0.76, 1.0]
ReLU     : [0.0, 0.0, 0.0, 1.0, 3.0]
소프트맥스: [0.0, 0.02, 0.04, 0.11, 0.83]
```

직접 해 보기 `predicted`를 전부 0(모두 정상)으로 바꾸면 정확도와 재현율은 각각 얼마가 되나요? 정확도만 믿으면 안 되는 이유를 말해 보세요.

확인 문제

06-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 손글씨 숫자 0~9를 맞히는 것은 다중 클래스 분류다. (O / X) 이름 _____

2 층을 많이 쌓기만 하면 활성화 함수 없이도 복잡한 곡선을 표현할 수 있다. (O / X) 이름 _____

3 "진짜 스팸 중에서 찾아낸 비율"은 무엇인가요? 이름 _____

① 정확도 ② 정밀도 ③ 재현율

4 훈련 데이터에서는 잘 맞히는데 새 데이터에서 못 맞히는 상태는? 이름 _____

① 과적합 ② 과소 적합 ③ 정규화 ④ 회귀

5 다중 분류 출력층에서 점수를 합이 1인 확률로 바꾸는 함수는 _____이다. 이름 _____

6 실습 결과에서 시그모이드에 0을 넣으면 _____이 나온다. 이름 _____

7 공항 보안 검색대 비유로 정밀도와 재현율의 차이를 설명해 보세요. 이름 _____

원문: 06-01 머신 러닝 기본 개념 정리하기 — https://wikidocs.net/60021

06-02 퍼셉트론

핵심 한 줄

퍼셉트론은 입력에 가중치를 곱해 더하고 기준을 넘으면 1을 내는 인공 뉴런이다. 한 층으로는 XOR을 못 풀지만, 층을 쌓으면 풀 수 있다.

왜 배우나

퍼셉트론은 모든 신경망의 **벽돌 한 장**입니다. GPT도 결국 이 벽돌을 엄청나게 많이 쌓은 것입니다. 그리고 "벽돌 한 장으로는 못 푸는 문제(XOR)가 있다"는 발견이 층을 쌓는 다층 퍼셉트론, 즉 딥러닝의 출발점이 되었습니다.

1) 뉴런 하나의 일

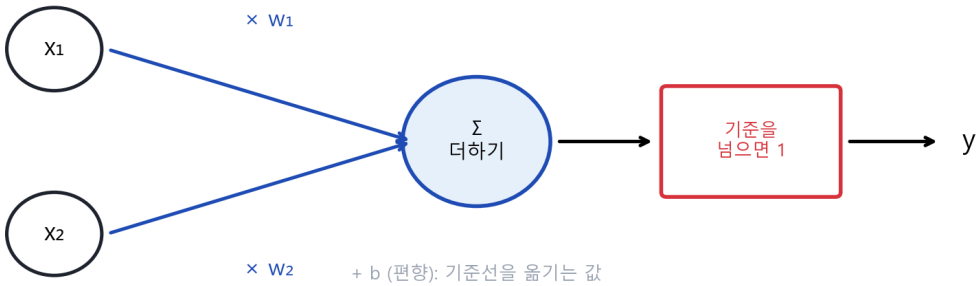


그림 4 퍼셉트론: 입력마다 가중치를 곱해 더하고(Σ), 기준을 넘으면 1, 아니면 0을 내보낸다

가중치 w는 "이 입력이 얼마나 중요한가"이고, 편향 b는 "얼마나 쉽게 1을 내보낼까"를 정합니다. 이 구조는 04장의 로지스틱 회귀와 거의 같습니다. 다른 점은 마지막에 계단처럼 0 또는 1로 딱 자른다는 것뿐입니다.

비유

점심 메뉴 투표

"오늘 짜장면 먹을까?"를 정할 때, 팀장 의견은 크게(가중치 0.6), 막내 의견은 작게(0.2) 반영하고, 합계가 기준을 넘으면 짜장면으로 결정합니다. 누구 말을 얼마나 반영할지가 가중치이고, 원래 짜장면을 얼마나 좋아하는지가 편향입니다.

각 사람의 의견	= 입력 x
의견 반영 비율	= 가중치 w
짜장면 결정 여부	= 출력 y (0 또는 1)

2) 단층 퍼셉트론의 한계: XOR

단층 퍼셉트론은 AND(둘 다 1일 때만 1), OR(하나라도 1이면 1), NAND(AND의 반대) 게이트를 만들 수 있습니다. 그런데 **XOR**(둘이 다를 때만 1)은 아무리 가중치를 바꿔도 만들 수 없습니다.

단층 퍼셉트론은 **직선 하나로만** 나눌 수 있는데, XOR의 빨간 점과 흰 점은 직선 하나로 나눌 수 없기 때문입니다.

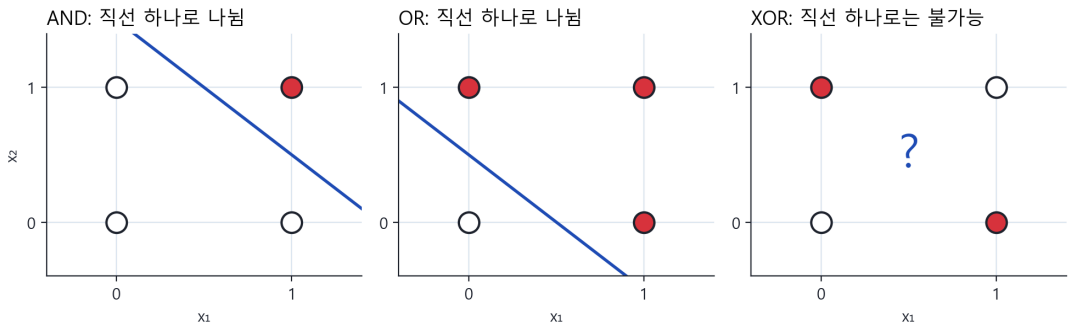


그림 5 빨간 점(출력 1)과 흰 점(출력 0)을 직선 하나로 나누기. AND와 OR은 되지만 XOR은 불가능하다

3) 층을 쌓으면 된다: 다층 퍼셉트론

NAND와 OR의 결과를 다시 AND에 넣으면 XOR이 됩니다. 즉, 퍼셉트론을 **두 층으로 쌓으면** 한 층으로 못 하던 일을 할 수 있습니다. 입력층과 출력층 사이의 층을 **은닉층**이라고 하고, 은닉층이 있는 구조를 **다층 퍼셉트론(MLP)**이라고 합니다. 은닉층이 2개 이상이면 **심층 신경망(DNN)**이라고 부르고, 이것을 학습시키는 것이 **딥러닝**입니다.

실습 2 논리 게이트를 퍼셉트론으로 만들기

```
# 퍼셉트론 = 입력 × 가중치를 모두 더하고, 기준(임계값)을 넘으면 1
def perceptron(x1, x2, w1, w2, b):
    return 1 if w1 * x1 + w2 * x2 + b > 0 else 0

def AND(x1, x2):  return perceptron(x1, x2, 0.5, 0.5, -0.7)
def OR(x1, x2):   return perceptron(x1, x2, 0.6, 0.6, -0.2)
def NAND(x1, x2): return perceptron(x1, x2, -0.5, -0.5, 0.7)

print("x1 x2 | AND OR NAND | XOR(정답)")
for x1, x2 in [(0, 0), (0, 1), (1, 0), (1, 1)]:
    print(f" {x1}  {x2} |  {AND(x1, x2)}   {OR(x1, x2)}   {NAND(x1, x2)}  |
{x1 ^ x2}")

# 층을 하나 더 쌓으면 XOR도 된다: XOR = AND(NAND, OR)
print("두 층으로 만든 XOR:", [AND(NAND(a, b), OR(a, b)) for a, b in [(0, 0),
(0, 1), (1, 0), (1, 1)]])
```

실행 결과

```
x1 x2 | AND OR NAND | XOR(정답)
 0  0 |  0   0   1  |   0
 0  1 |  0   1   1  |   1
 1  0 |  0   1   1  |   1
 1  1 |  1   1   0  |   0
두 층으로 만든 XOR: [0, 1, 1, 0]
```

직접 해 보기 AND의 편향 -0.7을 -0.2로 바꾸면 결과가 어떤 게이트와 같아지나요? 편향이 "얼마나 쉽게 1을 내는가"라는 뜻이 보이나요?

06-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 퍼셉트론은 입력과 가중치를 곱해 더한 값이 기준을 넘으면 1을 출력한다. (O / X) 이름 _____

2 단층 퍼셉트론으로 XOR 게이트를 만들 수 있다. (O / X) 이름 _____

3 XOR이 출력 1을 내는 경우는? 이름 _____

① 두 입력이 모두 1일 때 ② 두 입력이 서로 다를 때 ③ 두 입력이 모두 0일 때

4 입력층과 출력층 사이에 있는 층의 이름은? 이름 _____

① 은닉층 ② 편향층 ③ 활성층 ④ 정답층

5 단층 퍼셉트론은 _____ 하나로만 데이터를 나눌 수 있다. 이름 _____

6 실습에서 XOR은 AND(_____, OR)로 만들었다. 이름 _____

7 점심 메뉴 투표 비유로 가중치와 편향을 설명해 보세요. 이름 _____

원문: 06-02 퍼셉트론 — https://wikidocs.net/60680

06-03 XOR 문제: 단층 퍼셉트론과 다층 퍼셉트론

핵심 한 줄

같은 XOR 데이터를 파이토치로 학습시키면, 단층은 끝까지 못 풀고 은닉층을 쌓은 다층 퍼셉트론은 정확히 풀어낸다.

왜 배우나

06-02에서는 사람이 가중치를 직접 정했습니다. 이번에는 **모델이 스스로 가중치를 찾게** 합니다. 단층과 다층을 같은 조건에서 학습시켜 비교하면, "층을 쌓아야 한다"는 말이 실제 숫자로 확인됩니다.

1) 단층으로 학습해 보기

`nn.Linear(2, 1)` 뒤에 시그모이드를 붙인 모델은 04장의 로지스틱 회귀와 같습니다. 1만 번을 학습시켜도 네 입력 모두에 0.5를 내놓습니다. "둘 중 뭔지 모르겠다"며 포기한 것입니다. 손실도 0.69에서 더 내려가지 않습니다.

2) 은닉층을 쌓아 학습해 보기

은닉층 3개(각 뉴런 10개)를 쌓으면 손실이 거의 0이 되고 예측이 정답 [0, 1, 1, 0]과 같아집니다. 아래 그림처럼 다층 모델은 **굽은 경계**를 만들어 빨간 점들만 한 영역에 묶습니다.

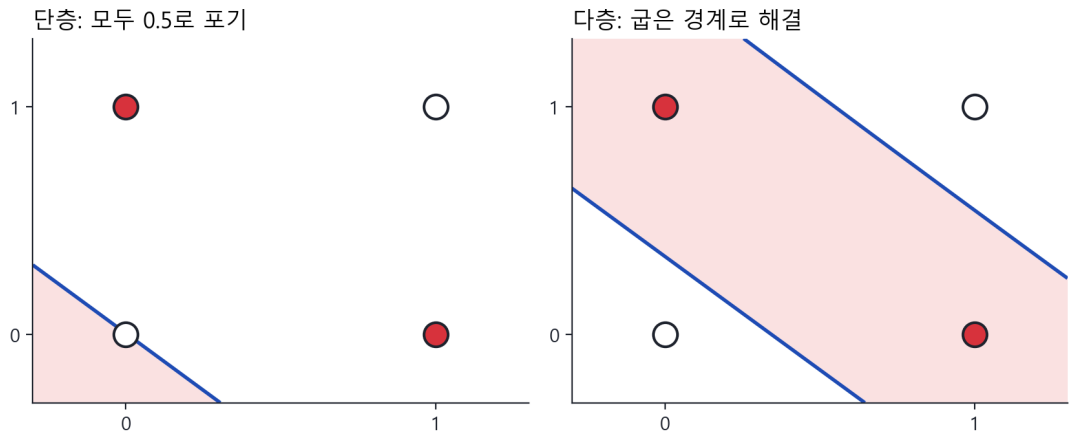


그림 6 학습된 모델이 출력 1이라고 판단하는 영역(분홍). 단층은 직선 경계라 실패하고, 다층은 띠 모양 경계로 성공한다

비유

종이 접기

평평한 종이 위의 점들을 가위로 한 번만 곧게 잘라 나눠야 한다면 XOR처럼 엇갈린 점들은 나눌 수 없습니다. 하지만 종이를 한 번 접으면 엇갈린 점들이 겹쳐지고, 그 상태에서는 한 번의 직선 가위질로 나눌 수 있습니다. 은닉층이 하는 일이 바로 이 "접기"입니다.

종이 접기 = 은닉층의 변환

곧은 가위질 = 마지막 출력층의 직선 경계

실습 3

단층과 다층으로 XOR 학습시키기

```python
import torch
import torch.nn as nn

torch.manual_seed(777)
X = torch.FloatTensor([[0, 0], [0, 1], [1, 0], [1, 1]])
Y = torch.FloatTensor([[0], [1], [1], [0]])      # XOR 정답

def train(model, epochs=10001):
    criterion = nn.BCELoss()                     # 0/1 분류용 손실
    optimizer = torch.optim.SGD(model.parameters(), lr=1)
    for step in range(epochs):
        cost = criterion(model(X), Y)
        optimizer.zero_grad()
        cost.backward()
        optimizer.step()
    return cost.item()

# (1) 단층 퍼셉트론: 입력 → 출력 한 층
single = nn.Sequential(nn.Linear(2, 1), nn.Sigmoid())
print("단층 손실:", round(train(single), 4))
print("단층 예측:", single(X).detach().round(decimals=2).squeeze().tolist())

# (2) 다층 퍼셉트론: 은닉층 3개(각 10개 뉴런)
multi = nn.Sequential(nn.Linear(2, 10), nn.Sigmoid(),
                      nn.Linear(10, 10), nn.Sigmoid(),
                      nn.Linear(10, 10), nn.Sigmoid(),
                      nn.Linear(10, 1), nn.Sigmoid())
print("다층 손실:", round(train(multi), 4))
print("다층 예측:", multi(X).detach().round(decimals=2).squeeze().tolist())
print("정답     :", Y.squeeze().tolist())
```

실행 결과

```
단층 손실: 0.6931
단층 예측: [0.5, 0.5, 0.5, 0.5]
다층 손실: 0.0001
다층 예측: [0.0, 1.0, 1.0, 0.0]
정답     : [0.0, 1.0, 1.0, 0.0]
```

직접 해 보기 다층 모델의 은닉층을 1개로 줄이면(`nn.Linear(2, 10), nn.Sigmoid(), nn.Linear(10, 1), nn.Sigmoid()`) 여전히 풀 수 있을까요?

06-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 실습에서 단층 모델은 네 입력 모두에 약 0.5를 예측했다. (O / X) 이름 _____

2 다층 퍼셉트론은 직선이 아닌 굽은 경계를 만들 수 있다. (O / X) 이름 _____

3 XOR처럼 0과 1을 맞히는 문제에 실습에서 쓴 손실 함수는? 이름 _____

① MSE ② BCELoss ③ 정확도

4 단층 모델의 손실이 0.69에서 멈춘 이유로 가장 알맞은 것은? 이름 _____

① 학습률이 너무 작아서 ② 직선 하나로는 XOR을 나눌 수 없어서 ③ 데이터가 너무 많아서

5 실습의 다층 모델에는 은닉층이 _____개 있다. 이름 _____

6 여러 층을 순서대로 이어 붙일 때 쓴 파이토치 도구는 nn. _____이다. 이름 _____

7 종이 접기 비유로 은닉층이 하는 일을 설명해 보세요. 이름 _____

원문: 06-03 XOR 문제 구현하기 — https://wikidocs.net/61010

06-04 역전파

핵심 한 줄 **역전파는 출력의 오차를 뒤에서 앞으로 거슬러 보내며, 각 가중치가 오차에 얼마나 책임이 있는지(기울기)를 계산하는 방법이다.**

왜 배우나 03장의 경사하강법은 "기울기를 알면 가중치를 고칠 수 있다"였습니다. 그런데 층이 여러 개면 앞쪽 가중치의 기울기는 어떻게 구할까요? 그 답이 역전파입니다. 파이토치의 `loss.backward()` 한 줄이 실제로 하는 일이 바로 이것입니다.

1) 순전파: 앞으로 계산하기

교재 예제는 입력 2개, 은닉 뉴런 2개, 출력 2개인 작은 신경망입니다. 입력 [0.1, 0.2]에 가중치를 곱하고 시그모이드를 통과시키며 앞으로 계산하면 출력은 [0.609, 0.664]가 됩니다. 목표는 [0.4, 0.6]이므로 오차가 생깁니다. 이렇게 입력에서 출력 방향으로 계산하는 것이 **순전파**입니다.

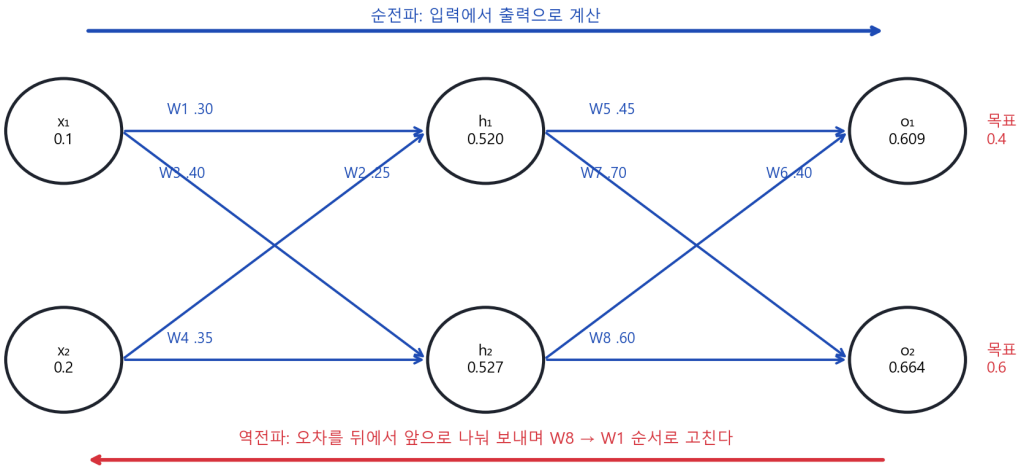


그림 7 교재 예제의 신경망. 순전파(파랑)로 출력을 만들고, 역전파(빨강)로 오차의 책임을 뒤에서 앞으로 나눈다

2) 역전파: 뒤에서부터 책임 나누기

먼저 출력에 가까운 W5~W8의 기울기를 구합니다. 예를 들어 W5의 기울기는 세 가지를 곱해서 얻습니다.

$$\frac{\partial E}{\partial W_5} = \frac{\partial E}{\partial o_1} \times \frac{\partial o_1}{\partial z_3} \times \frac{\partial z_3}{\partial W_5}$$

(오차가 o_1에 얼마나 민감한가) × (o_1이 z_3에 얼마나 민감한가) × (z_3가 W_5에 얼마나 민감한가). 말로 하면: 영향이 전달되는 길을 따라 한 칸씩 "얼마나 민감한가"를 곱한다. 이것을 **연쇄 법칙(chain rule)**이라고 합니다.

계산하면 0.0259이고, 학습률 0.5로 고치면 W5는 0.45에서 0.437로 바뀝니다. 그다음 같은 방식으로 한 층 더 앞의 W1~W4를 고칩니다. 이때 뒤에서 이미 계산한 값을 재사용하므로, 층이

아무리 많아도 효율적으로 계산할 수 있습니다. 한 번 고친 뒤 다시 순전파하면 전체 오차가 0.02397에서 0.02324로 줄어듭니다.

비유

조별 과제 점수가 나빴을 때

발표 점수가 낮게 나오면, 먼저 발표자에게 "어디가 문제였나"를 묻고, 발표자는 그 원인을 자료 조사 담당과 PPT 담당에게 다시 나눠 전달합니다. 결과에서 출발해 한 단계씩 거슬러 올라가며 각자의 책임만큼 고칠 점을 알려 주는 것이 역전파입니다.

낮은 발표 점수	= 출력의 오차
발표자 → 담당자 순서로 전달	= 출력층 → 은닉층 방향 (역방향)
각자의 책임 크기	= 가중치마다의 기울기

실습 4

교재의 손 계산을 파이토치로 확인하기

```python
import torch

# 교재와 같은 숫자: 입력 2개, 은닉 뉴런 2개, 출력 2개
x = torch.tensor([0.1, 0.2])
target = torch.tensor([0.4, 0.6])
W_in = torch.tensor([[0.30, 0.25],     # W1, W2 → h1
                     [0.40, 0.35]],    # W3, W4 → h2
                    requires_grad=True)
W_out = torch.tensor([[0.45, 0.40],    # W5, W6 → o1
                      [0.70, 0.60]],   # W7, W8 → o2
                     requires_grad=True)

def forward():
    h = torch.sigmoid(W_in @ x)        # 순전파 1: 입력 → 은닉층
    o = torch.sigmoid(W_out @ h)       # 순전파 2: 은닉층 → 출력
    return o, (0.5 * (target - o) ** 2).sum()

o, E = forward()
print("출력 o1, o2:", [round(v, 6) for v in o.tolist()])
print("전체 오차  :", round(E.item(), 8))

E.backward()                           # 역전파: 모든 가중치의 기울기를 한 번에
print("dE/dW5     :", round(W_out.grad[0, 0].item(), 8))
print("dE/dW1     :", round(W_in.grad[0, 0].item(), 8))

with torch.no_grad():                  # 경사하강법 한 걸음 (학습률 0.5)
    W_in -= 0.5 * W_in.grad
    W_out -= 0.5 * W_out.grad
print("새 W5      :", round(W_out[0, 0].item(), 8))
print("다시 순전파한 오차:", round(forward()[1].item(), 8))
```

실행 결과

```
출력 o1, o2: [0.609446, 0.663845]
전체 오차  : 0.0239719
dE/dW5     : 0.02592285
dE/dW1     : 0.00080888
새 W5      : 0.43703857
다시 순전파한 오차: 0.02323634
```

직접 해 보기 교재의 손 계산 결과(dE/dW5 = 0.02592286, 새 W5 = 0.43703857)와 비교해 보세요. `E.backward()` 한 줄이 그 긴 계산을 대신해 줍니다.

확인 문제

06-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 순전파는 입력에서 출력 방향으로 계산하는 것이다. (O / X) 이름 _____

2 역전파에서는 입력층에 가까운 가중치부터 먼저 고친다. (O / X) 이름 _____

3 역전파가 각 가중치에 대해 계산하는 것은? 이름 _____

① 입력값 ② 기울기(오차에 대한 책임) ③ 데이터 개수

4 파이토치에서 역전파를 실행하는 코드는? 이름 _____

① loss.backward() ② model.forward() ③ torch.sigmoid() ④ optimizer.zero_grad()

5 영향이 전달되는 길을 따라 민감도를 곱해 기울기를 구하는 규칙을 _____ 법칙이라고 한다. 이름 _____

6 실습에서 한 번 고친 뒤 전체 오차는 0.02397에서 _____로 줄었다. 이름 _____

7 조별 과제 비유로 역전파가 왜 "역(거꾸로)"인지 설명해 보세요. 이름 _____

원문: 06-04 역전파 — https://wikidocs.net/60682

06-05 다층 퍼셉트론으로 손글씨 분류하기

핵심 한 줄

8×8 손글씨 그림을 64개 숫자로 펴서 다층 퍼셉트론에 넣으면, 0~9 중 어떤 숫자인지 분류할 수 있다.

왜 배우나

지금까지 배운 것(다층 퍼셉트론, 활성화 함수, 역전파, 경사하강법)을 처음으로 **진짜 데이터**에 써 보는 소챕터입니다. "그림을 숫자로 바꿔서 신경망에 넣는다"는 생각은 나중에 "문장을 숫자로 바꿔서 넣는다"로 그대로 이어집니다.

1) 그림도 숫자다

사이킷런의 손글씨 데이터는 1,797장이고, 한 장은 8×8 칸에 0~16의 진하기가 적힌 표입니다. 이 표를 한 줄로 펴면 숫자 64개가 되고, 이것이 신경망의 입력이 됩니다. 정답은 0~9 중 하나이므로 출력은 10개입니다.

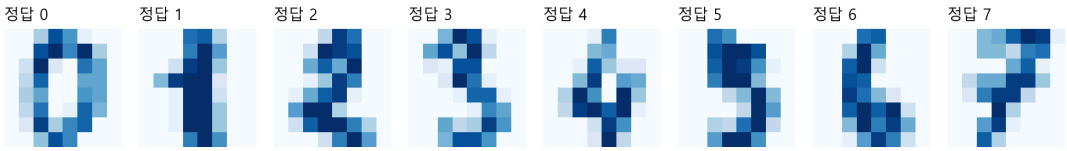


그림 8 사이킷런 손글씨 데이터의 첫 8장. 칸이 진할수록 큰 숫자다

2) 모델: 64 → 32 → 16 → 10

`nn.Sequential`로 층을 차례로 쌓고 사이사이에 ReLU를 넣습니다. 여러 개 중 하나를 고르는 문제라서 손실 함수는 **CrossEntropyLoss**를 쓰고, 옵티마이저는 SGD보다 보통 더 빠르게 학습하는 **Adam**을 씁니다. 예측은 출력 10개 중 가장 큰 값의 위치(`argmax`)입니다.

비유

모자이크 퍼즐

멀리서 보면 숫자인데 가까이서 보면 색칠된 칸 64개일 뿐인 모자이크를 떠올려 보세요. 신경망은 "어느 칸이 진하면 어떤 숫자일 가능성이 높다"는 규칙을 수많은 예시에서 스스로 찾아냅니다. 사람이 규칙을 알려 주지 않는다는 점이 중요합니다.

모자이크 칸 64개	= 입력 특성 64개
칸과 숫자 사이의 규칙	= 학습된 가중치
"이건 7이다"	= 출력 10개 중 가장 큰 값

실습 5 손글씨 숫자 분류기 학습

```
import torch
import torch.nn as nn
from sklearn.datasets import load_digits

torch.manual_seed(0)
digits = load_digits()                          # 8×8 손글씨 숫자 1,797장 (다운로드 없음)
X = torch.tensor(digits.data, dtype=torch.float32)   # (1797, 64): 그림을 한 줄로 편 것
Y = torch.tensor(digits.target, dtype=torch.int64)   # 0~9 정답
print("데이터 모양:", tuple(X.shape), " 첫 정답:", Y[0].item())

model = nn.Sequential(
    nn.Linear(64, 32), nn.ReLU(),               # 입력 64칸 → 은닉 32
    nn.Linear(32, 16), nn.ReLU(),               # 은닉 32 → 은닉 16
    nn.Linear(16, 10)                           # 은닉 16 → 출력 10 (숫자 0~9)
)
loss_fn = nn.CrossEntropyLoss()                 # 여러 개 중 하나를 고르는 분류용 손실
optimizer = torch.optim.Adam(model.parameters())

for epoch in range(100):
    loss = loss_fn(model(X), Y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch % 25 == 0 or epoch == 99:
        acc = (model(X).argmax(dim=1) == Y).float().mean().item()
        print(f"Epoch {epoch:3d}  Loss: {loss.item():.4f}  정확도: {acc:.3f}")
```

실행 결과

```
데이터 모양: (1797, 64)  첫 정답: 0
Epoch   0  Loss: 2.4023  정확도: 0.066
Epoch  25  Loss: 1.5495  정확도: 0.582
Epoch  50  Loss: 0.7318  정확도: 0.845
Epoch  75  Loss: 0.3586  정확도: 0.923
Epoch  99  Loss: 0.2146  정확도: 0.957
```

직접 해 보기 은닉층의 ReLU를 빼면 정확도가 어떻게 변하나요? 06-01의 "활성화 함수가 없으면 결국 직선"과 연결해 보세요.

확인 문제

06-05 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 8×8 그림을 한 줄로 펴면 입력은 64개의 숫자가 된다. (O / X) 이름 _____

2 숫자 0~9를 분류하므로 출력층의 크기는 1이다. (O / X) 이름 _____

3 여러 개 중 하나를 고르는 분류에 실습에서 쓴 손실 함수는? 이름 _____

① MSE ② CrossEntropyLoss ③ BCELoss

4 출력 10개 중 가장 큰 값의 위치를 찾는 함수는? 이름 _____

① argmax ② sum ③ mean ④ flatten

5 실습 모델의 층 크기는 64 → 32 → _____ → 10이다. 이름 _____

6 실습에서 100번 학습한 뒤 정확도는 약 _____%였다. 이름 _____

7 모자이크 퍼즐 비유로 "그림도 숫자다"를 설명해 보세요. 이름 _____

원문: 06-05 다층 퍼셉트론으로 손글씨 분류하기 — https://wikidocs.net/61046

06-06 다층 퍼셉트론으로 MNIST 분류하기

핵심 한 줄

더 크고 유명한 손글씨 데이터 MNIST(28×28, 7만 장)를 훈련용과 테스트용으로 나누고, 미니 배치로 학습해 처음 보는 그림의 정확도로 평가한다.

왜 배우나

06-05는 훈련한 데이터로 정확도를 쟀습니다. 그건 외운 문제로 시험을 본 것과 같습니다. 이번에는 **처음 보는 테스트 데이터**로 평가하고, 데이터가 많아서 03-04의 **미니 배치와 DataLoader**를 씁니다. 실제 프로젝트의 학습 흐름과 똑같은 모양입니다.

1) MNIST 데이터

MNIST는 딥러닝의 "Hello, World"로 불리는 데이터입니다. 28×28 = 784칸의 흑백 손글씨 숫자이고, 훈련용 6만 장과 테스트용 1만 장으로 나뉩니다. 교재는 사이킷런의 `fetch_openml`로 받지만, 이 자료에서는 08장과 같은 `torchvision` 방식으로 받습니다. 데이터는 같습니다.



그림 9 MNIST 훈련 데이터의 첫 8장. 06-05보다 해상도가 높아 글씨 모양이 선명하다

2) 학습 흐름

1. `nn.Flatten()`으로 28×28 그림을 784칸 한 줄로 폅니다.
2. 784 → 100 → 100 → 10 모델을 만듭니다.
3. DataLoader로 64장씩 나눠 3 에포크 학습합니다.
4. `torch.no_grad()`로 기울기 계산을 끄고, 테스트 1만 장의 정확도를 잽니다.

비유

모의고사와 수능

문제집(훈련 데이터)으로 아무리 공부해도, 실력은 처음 보는 수능 문제(테스트 데이터)로 확인해야 합니다. 그래서 테스트 데이터는 학습에 절대 쓰지 않고 마지막 평가용으로만 남겨 둡니다.

문제집 = 훈련 데이터 6만 장
수능 = 테스트 데이터 1만 장
수능 점수 = 테스트 정확도

실습 6 MNIST 분류기 학습과 테스트 평가

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
import torchvision.datasets as dsets
import torchvision.transforms as transforms

torch.manual_seed(1)
# MNIST: 28×28 손글씨 숫자. 훈련 6만 장, 테스트 1만 장 (처음 한 번 다운로드)
train = dsets.MNIST(root="MNIST_data/", train=True, transform=transforms.To
Tensor(), download=True)
test = dsets.MNIST(root="MNIST_data/", train=False, transform=transforms.To
Tensor(), download=True)
loader = DataLoader(train, batch_size=64, shuffle=True)

model = nn.Sequential(
    nn.Flatten(),                          # 28×28 그림을 784칸 한 줄로 펴기
    nn.Linear(784, 100), nn.ReLU(),
    nn.Linear(100, 100), nn.ReLU(),
    nn.Linear(100, 10))
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(3):
    for x, y in loader:
        loss = loss_fn(model(x), y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch + 1}  마지막 배치 Loss: {loss.item():.4f}")

with torch.no_grad():                      # 평가할 때는 기울기 계산 끄기
    x_test = test.data.float().div(255).unsqueeze(1)
    acc = (model(x_test).argmax(dim=1) == test.targets).float().mean()
print("테스트 정확도:", round(acc.item(), 4))
```

실행 결과

```
Epoch 1  마지막 배치 Loss: 0.2200
Epoch 2  마지막 배치 Loss: 0.0719
Epoch 3  마지막 배치 Loss: 0.0210
테스트 정확도: 0.9699
```

직접 해 보기 에포크를 3에서 10으로 늘리면 테스트 정확도가 계속 오를까요? 06-01의 과적합 그래프를 떠올려 보세요. (코랩 CPU에서 에포크당 약 10~20초)

확인 문제

06-06 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 테스트 데이터는 학습에 쓰지 않고 마지막 평가에만 쓴다. (O / X) 이름 _____

2 MNIST 그림 한 장을 한 줄로 펴면 784개의 숫자가 된다. (O / X) 이름 _____

3 평가할 때 기울기 계산을 끄는 코드는? 이름 _____

① torch.no_grad() ② loss.backward() ③ optimizer.step()

4 MNIST의 훈련 데이터 개수는? 이름 _____

① 1,797장 ② 1만 장 ③ 6만 장 ④ 100만 장

5 그림을 한 줄로 펴 주는 층은 nn._____이다. 이름 _____

6 실습의 테스트 정확도는 약 _____%였다. 이름 _____

7 모의고사와 수능 비유로 훈련 데이터와 테스트 데이터를 나누는 이유를 설명해 보세요. 이름 _____

원문: 06-06 다층 퍼셉트론으로 MNIST 분류하기 — https://wikidocs.net/61073

06-07 과적합을 막는 방법들

핵심 한 줄 **과적합은 데이터 늘리기, 모델 줄이기, 가중치 규제, 드롭아웃으로 막는다.**

왜 배우나 층을 쌓을수록 모델은 똑똑해지지만, 동시에 훈련 데이터를 **외워 버리기도 쉬워집니다**. 06-01에서 본 과적합 그래프의 "다시 올라가는 구간"을 늦추거나 없애는 방법들이 이 소챕터의 내용이고, 앞으로 나올 거의 모든 모델 코드에 들어갑니다.

1) 네 가지 방법

방법	하는 일	파이토치에서
데이터 늘리기	외울 수 없을 만큼 다양한 예시를 본다. 그림을 돌리거나 문장을 번역했다 되돌리는 증강도 포함	데이터 준비 단계
모델 줄이기	층 수나 뉴런 수를 줄여 외울 공간을 줄인다	nn.Linear 크기 줄이기
가중치 규제	가중치가 너무 커지지 않게 손실에 벌점을 더한다 (L2)	`weight_decay=1e-5`
드롭아웃	학습할 때마다 뉴런 일부를 무작위로 끈다	`nn.Dropout(p=0.5)`

2) 드롭아웃

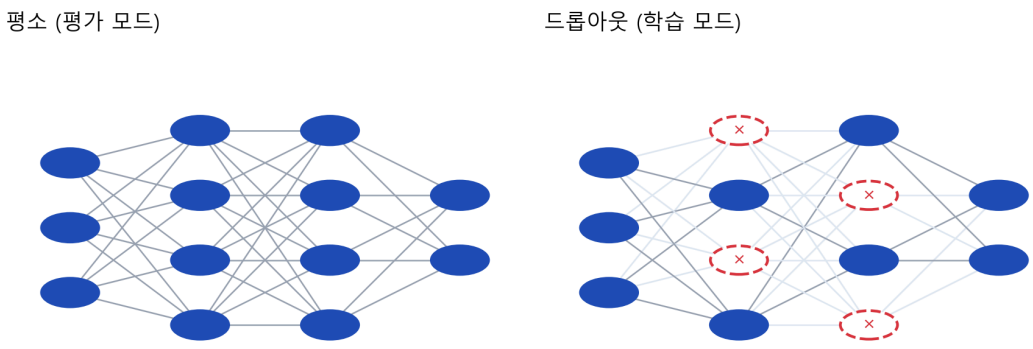


그림 10 드롭아웃은 학습할 때마다 다른 뉴런을 끈다(×). 평가할 때는 모든 뉴런을 쓴다

매번 다른 뉴런이 꺼지니 모델은 특정 뉴런 몇 개에만 의지할 수 없고, 여러 뉴런에 지식을 골고루 나눠 담게 됩니다. **학습할 때만** 끄고 평가할 때는 모두 켜야 하므로, `model.train()` 과 `model.eval()` 로 모드를 바꿔 줍니다. 실습에서 꺼지지 않은 뉴런의 값이 1이 아니라 2가 되는 것은, 절반이 꺼진 만큼 나머지를 2배로 키워 전체 크기를 맞추기 때문입니다.

비유

매번 다른 멤버가 빠지는 팀 연습

축구팀이 연습할 때마다 무작위로 선수 몇 명을 빼고 경기하면, 특정 에이스에게만 공을 주는 습관이 사라지고 모두가 여러 역할을 할 줄 알게 됩니다. 실제 경기에서는 전원이 뛰니 훨씬 단단한 팀이 됩니다.

연습마다 빠지는 선수 = 드롭아웃으로 꺼지는 뉴런
실제 경기 = 평가 모드 (모두 켜짐)
에이스 의존 습관 = 과적합

실습 7

드롭아웃의 학습 모드와 평가 모드

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
dropout = nn.Dropout(p=0.5)       # 학습 중에 뉴런의 절반을 무작위로 끈다
x = torch.ones(10)

dropout.train()                   # 학습 모드
print("학습 모드 1회:", dropout(x).tolist())
print("학습 모드 2회:", dropout(x).tolist())   # 매번 꺼지는 뉴런이 다름
dropout.eval()                    # 평가 모드: 아무것도 끄지 않음
print("평가 모드    :", dropout(x).tolist())

# 가중치 규제(L2)는 옵티마이저의 weight_decay 한 줄로
model = nn.Linear(10, 1)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)
print("weight_decay:", optimizer.defaults["weight_decay"])
```

실행 결과

```
학습 모드 1회: [0.0, 0.0, 2.0, 0.0, 0.0, 0.0, 2.0, 2.0, 0.0, 2.0]
학습 모드 2회: [2.0, 2.0, 2.0, 0.0, 2.0, 2.0, 2.0, 2.0, 0.0, 0.0]
평가 모드    : [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
weight_decay: 1e-05
```

직접 해 보기 `p=0.5`를 `0.2`로 바꾸면 꺼지는 개수와 살아남은 값의 크기가 어떻게 바뀌나요?

06-07 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 드롭아웃은 평가할 때도 뉴런 일부를 끈다. (O / X) 이름 _____

2 데이터를 늘리는 것은 과적합을 막는 방법 중 하나다. (O / X) 이름 _____

3 파이토치에서 L2 가중치 규제를 거는 방법은? 이름 _____

① 옵티마이저의 weight_decay ② nn.Flatten ③ torch.no_grad

4 평가 모드로 바꾸는 코드는? 이름 _____

① model.train() ② model.eval() ③ model.backward()

5 실습의 드롭아웃은 뉴런의 _____%를 무작위로 끈다. 이름 _____

6 절반이 꺼지면 살아남은 값은 _____배로 커진다. 이름 _____

7 팀 연습 비유로 드롭아웃이 과적합을 막는 이유를 설명해 보세요. 이름 _____

원문: 06-07 과적합을 막는 방법들 — https://wikidocs.net/60751

06-08 기울기 소실과 폭주

핵심 한 줄

층이 깊으면 역전파되는 기울기가 앞쪽에서 거의 0이 되거나(소실) 너무 커진다(폭주). ReLU, 좋은 초기화, 배치 정규화로 막는다.

왜 배우나

역전파는 뒤에서 앞으로 기울기를 곱해 가며 전달합니다(06-04). 1보다 작은 수를 계속 곱하면 0에 가까워지고, 큰 수를 곱하면 폭발합니다. 그러면 앞쪽 층은 거의 배우지 못합니다. 07장의 RNN, 그리고 그 뒤의 LSTM과 트랜스포머가 등장한 이유도 이 문제와 깊이 관련되어 있습니다.

1) 왜 기울기가 사라지나

시그모이드의 기울기는 아무리 커도 0.25입니다. 10층을 거슬러 오면 0.25를 열 번 곱하는 셈이라 0.000001보다 작아집니다. 실습에서 시그모이드 10층의 맨 앞 층 기울기는 맨 뒤 층의 약 100억분의 1(1.3e-10배)이었습니다.

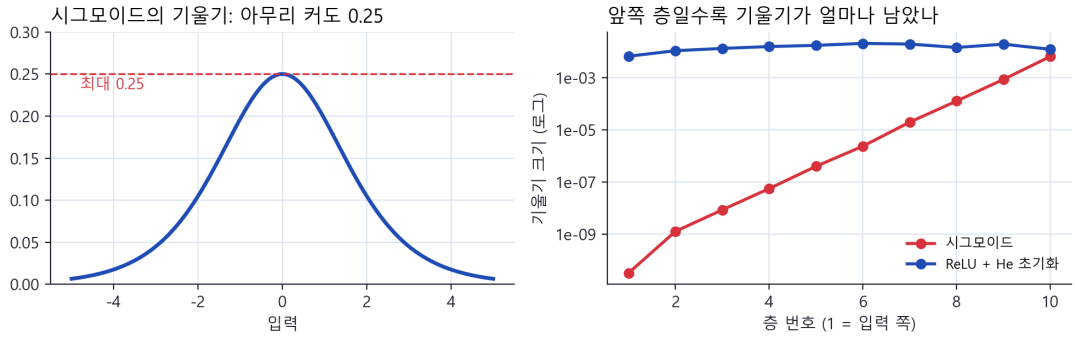


그림 11 왼쪽: 시그모이드의 기울기는 최대 0.25. 오른쪽: 10층 신경망에서 층마다 남은 기울기. 시그모이드는 앞쪽으로 갈수록 급격히 사라진다

2) 해결 방법

- **ReLU 계열 사용**: 양수 구간의 기울기가 1이라 곱해도 줄지 않습니다. 은닉층에는 시그모이드 대신 ReLU를 씁니다.
- **가중치 초기화**: 처음 가중치를 아무렇게나 정하지 않고, 층을 지나도 값의 크기가 유지되도록 정합니다. 시그모이드·tanh에는 **세이비어(Xavier) 초기화**, ReLU에는 **He 초기화**가 잘 맞습니다.
- **배치 정규화**: 각 층의 입력을 미니 배치마다 평균 0, 표준편차 1로 맞춥니다. 층마다 입력 분포가 출렁이는 문제를 줄여 학습이 안정됩니다. 배치 크기가 너무 작거나 RNN처럼 길이가 다른 데이터에는 **층 정규화**를 씁니다.
- **기울기 클리핑**: 기울기가 폭주하면 정해 둔 크기 이상으로 커지지 않게 자릅니다. RNN에서 자주 씁니다.

비유

말 전달 게임

열 명이 줄을 서서 귓속말로 말을 전하면, 뒤로 갈수록 말이 작아지고 흐려져 마지막 사람은 거의 못 알아듣습니다. 반대로 한 명이 매번 과장하면 말이 점점 부풀려집니다. 각 사람이 "들은 크기 그대로" 전하도록 규칙을 정하는 것이 ReLU, 초기화, 정규화가 하는 일입니다.

줄 선 사람들 = 층

점점 작아지는 말 = 기울기 소실

점점 부풀려지는 말 = 기울기 폭주

실습 8

10층 신경망에서 기울기가 얼마나 살아남나

```
import torch
import torch.nn as nn

torch.manual_seed(0)

def grad_ratio(activation, he_init=False, depth=10):
    layers = []
    for _ in range(depth):                          # 같은 크기의 층을 10개 쌓기
        linear = nn.Linear(32, 32)
        if he_init:                                 # He 초기화: ReLU에 맞춘 시작 가중치
            nn.init.kaiming_normal_(linear.weight, nonlinearity="relu")
        layers += [linear, activation()]
    model = nn.Sequential(*layers, nn.Linear(32, 1))
    model(torch.randn(64, 32)).pow(2).mean().backward()
    first = model[0].weight.grad.abs().mean().item()    # 맨 앞 층에 도착한 기울기
    last = model[-1].weight.grad.abs().mean().item()    # 맨 뒤 층의 기울기
    return first / last                                 # 뒤에서 앞으로 오며 얼마나 남았나

print(f"시그모이드          : 앞 층 기울기 = 뒤 층의 {grad_ratio(nn.Sigmoid):.1e}배")
print(f"ReLU                : 앞 층 기울기 = 뒤 층의 {grad_ratio(nn.ReLU):.1e}배")
print(f"ReLU + He 초기화    : 앞 층 기울기 = 뒤 층의 {grad_ratio(nn.ReLU, he_init=True):.1e}배")

# 배치 정규화: 층을 지날 때마다 값을 평균 0, 표준편차 1 근처로 맞춰 준다
bn = nn.BatchNorm1d(3)
h = torch.tensor([[10., 200., -5.], [12., 180., -7.], [14., 220., -3.]])
print("정규화 전 평균:", h.mean(dim=0).tolist())
out = bn(h)
print("정규화 후 평균:", [round(v, 2) + 0.0 for v in out.mean(dim=0).tolist()])
print("정규화 후 표준편차:", [round(v, 2) for v in out.std(dim=0, unbiased=False).tolist()])
```

```
실행 결과
시그모이드          : 앞 층 기울기 = 뒤 층의 1.3e-10배
ReLU                : 앞 층 기울기 = 뒤 층의 3.6e-05배
ReLU + He 초기화    : 앞 층 기울기 = 뒤 층의 1.4e-02배
정규화 전 평균: [12.0, 200.0, -5.0]
정규화 후 평균: [0.0, 0.0, 0.0]
정규화 후 표준편차: [1.0, 1.0, 1.0]
```

직접 해 보기 `depth=10`을 `3`으로 줄이면 시그모이드의 비율은 얼마나 나아지나요? 층이 깊을수록 문제가 커진다는 것을 확인해 보세요.

확인 문제

06-08 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 시그모이드의 기울기는 최대 0.25이다. (O / X) 이름 _____

2 기울기 소실이 생기면 출력층에 가까운 층이 가장 배우기 어렵다. (O / X) 이름 _____

3 은닉층에서 기울기 소실을 줄이기 위해 시그모이드 대신 주로 쓰는 것은? 이름 _____

① ReLU ② 소프트맥스 ③ MSE

4 ReLU와 잘 맞는 가중치 초기화 방법은? 이름 _____

① He 초기화 ② 0으로 초기화 ③ 세이비어 초기화만 가능

5 각 층의 입력을 미니 배치마다 평균 0, 표준편차 1로 맞추는 방법은 _____ 정규화다. 이름 _____

6 기울기가 너무 커지지 않게 잘라 내는 방법은 기울기 _____ 이다. 이름 _____

7 말 전달 게임 비유로 기울기 소실과 폭주를 설명해 보세요. 이름 _____

원문: 06-08 기울기 소실과 폭주 — https://wikidocs.net/61271

오늘의 요약

다시 보기



핵심 한 줄 모음

소챕터	핵심 한 줄
06-01	분류·회귀, 정확도·정밀도·재현율, 과적합, 활성화 함수(직선을 구부리는 장치)
06-02	퍼셉트론은 인공 뉴런. 한 층으로는 XOR을 못 풀지만 층을 쌓으면 풀 수 있다.
06-03	학습시켜 보면 단층은 0.5로 포기하고, 다층은 굽은 경계로 XOR을 정확히 푼다.
06-04	역전파는 오차를 뒤에서 앞으로 보내며 연쇄 법칙으로 가중치마다의 기울기를 구한다.
06-05	그림을 숫자로 펴서 넣으면 다층 퍼셉트론으로 손글씨를 분류할 수 있다.
06-06	훈련/테스트를 나누고 미니 배치로 학습해, 처음 보는 데이터의 정확도로 평가한다.
06-07	과적합은 데이터 늘리기, 모델 줄이기, 가중치 규제, 드롭아웃으로 막는다.
06-08	깊은 층의 기울기 소실·폭주는 ReLU, He 초기화, 배치 정규화, 클리핑으로 막는다.

다음

다음 자료 예고: 07장 순환 신경망

지금까지의 신경망은 입력을 한 번에 받습니다. 하지만 문장은 단어가 **순서대로** 나옵니다. 07장에서는 앞의 단어를 기억하면서 다음 단어를 읽는 순환 신경망(RNN)을 배웁니다. 오늘 본 기울기 소실이 RNN에서 다시 중요한 문제로 등장합니다.

정답과 해설

06-01 머신 러닝 기본 개념 정리하기

1. O — 10개 중 하나를 고르므로 다중 클래스 분류입니다.
2. X — 곱하고 더하기만 반복하면 결국 직선 하나와 같습니다. 활성화 함수가 있어야 곡선을 표현합니다.
3. ③ — 재현율 $= \dfrac{TP}{TP + FN}$
4. ①
5. 소프트맥스
6. 0.5
7. 예시 답: 멀쩡한 가방을 덜 열어 보게 하는 것이 정밀도, 위험물을 하나도 놓치지 않게 하는 것이 재현율이다. 하나를 높이면 다른 하나가 낮아지기 쉽다.

06-02 퍼셉트론

1. O
2. X — 직선 하나로 나눌 수 없어서 불가능합니다.
3. ②
4. ①
5. 직선
6. NAND
7. 예시 답: 누구 의견을 얼마나 반영할지가 가중치, 원래 그 메뉴를 얼마나 좋아하는지가 편향이다.

06-03 XOR 문제

1. O
2. O
3. ② — 0/1을 맞히는 이진 분류이므로 BCELoss를 씁니다.
4. ②
5. 3
6. Sequential
7. 예시 답: 엇갈린 점들은 평평한 종이에서는 직선으로 못 나누지만, 종이를 접으면(은닉층의 변환) 한 번의 직선으로 나눌 수 있다.

06-04 역전파

1. O
2. X — 출력층에 가까운 W5~W8부터 고치고, 그다음 W1~W4를 고칩니다.
3. ②
4. ①
5. 연쇄
6. 0.02324

7. 예시 답: 결과(발표 점수)에서 출발해 발표자, 담당자 순서로 거슬러 올라가며 책임을 나누기 때문에 "거꾸로"다.

06-05 다층 퍼셉트론으로 손글씨 분류하기

1. O
2. X — 숫자 10개의 점수를 내야 하므로 출력은 10입니다.
3. ②
4. ①
5. 16
6. 96 (실행 결과 0.957)
7. 예시 답: 숫자 그림은 진하기가 적힌 칸 64개이고, 신경망은 어느 칸이 진할 때 어떤 숫자인지 규칙을 스스로 찾는다.

06-06 다층 퍼셉트론으로 MNIST 분류하기

1. O
2. O — 28 × 28 = 784
3. ①
4. ③
5. Flatten
6. 97 (실행 결과 0.9699)
7. 예시 답: 문제집만 잘 푸는 것이 아니라 처음 보는 문제도 풀 수 있는지 확인해야 진짜 실력을 알 수 있다.

06-07 과적합을 막는 방법들

1. X — 학습할 때만 끄고, 평가할 때는 모두 켭니다.
2. O
3. ①
4. ②
5. 50
6. 2
7. 예시 답: 매번 다른 선수가 빠지면 특정 에이스에게만 의지하지 않고 모두가 고르게 실력을 쌓는다. 신경망도 특정 뉴런에 의존하지 않게 된다.

06-08 기울기 소실과 폭주

1. O
2. X — 입력층에 가까운(앞쪽) 층이 가장 배우기 어렵습니다.
3. ①
4. ①
5. 배치
6. 클리핑

7. 예시 답: 귓속말이 뒤로 갈수록 작아지는 것이 소실, 과장되며 커지는 것이 폭주다. 앞쪽 사람(앞쪽 층)은 제대로 된 말을 못 듣는다.

용어 정리

용어	쉬운 뜻
분류 / 회귀	정해진 종류 중 하나 맞히기 / 연속된 숫자 맞히기
정밀도 / 재현율	양성이라고 한 것 중 진짜 비율 / 진짜 양성 중 찾아낸 비율
과적합	훈련 데이터를 외워서 새 데이터에서 성능이 떨어지는 상태
활성화 함수	층의 출력을 구부려 직선이 아닌 관계를 표현하게 하는 함수 (ReLU, 시그모이드, 소프트맥스 등)
퍼셉트론	입력 × 가중치를 더해 기준을 넘으면 1을 내는 인공 뉴런
은닉층 / 다층 퍼셉트론 (MLP)	입력층과 출력층 사이의 층 / 은닉층이 있는 신경망
순전파 / 역전파	입력에서 출력으로 계산 / 오차를 뒤에서 앞으로 보내며 기울기 계산
연쇄 법칙	영향이 전달되는 길을 따라 민감도를 곱해 기울기를 구하는 규칙
CrossEntropyLoss / Adam	다중 분류용 손실 함수 / 많이 쓰는 옵티마이저
훈련 / 테스트 데이터	학습에 쓰는 데이터 / 마지막 평가에만 쓰는 처음 보는 데이터
드롭아웃 / 가중치 규제	학습 중 뉴런 일부를 끄기 / 가중치가 커지지 않게 벌점 주기
기울기 소실 / 폭주	역전파 중 기울기가 0에 가까워짐 / 너무 커짐
He 초기화 / 배치 정규화	ReLU에 맞춘 시작 가중치 / 층 입력을 평균 0, 표준편차 1로 맞추기